

Commented-Out Code and Dead Code

Commenting out code so it doesn't run is fine as a temporary measure. You might want to skip some lines to test other functionality, and commenting them out makes them easy to add back in later. But if commented-out code remains in place, it's a complete mystery why it was removed and under what condition it might ever be needed again. Consider the following example:

```
doSomething()  
#doAnotherThing()  
doSomeImportantTask()  
doAnotherThing()
```

This code prompts many unanswered questions: Why was `doAnotherThing()` commented out? Will we ever include it again? Why wasn't the second call to `doAnotherThing()` commented out? Were there originally two calls to `doAnotherThing()`, or was there one call that was moved after `doSomeImportantTask()`? Is there a reason we shouldn't remove the commented-out code? There are no readily available answers to these questions.

Dead code is code that is unreachable or logically can never run. For example, code inside a function but after a `return` statement, code in an `if` statement block with an always `False` condition, or code in a function that is never called is all dead code. To see this in practice, enter the following into the interactive shell:

```
>>> import random  
>>> def coinFlip():  
...     if random.randint(0, 1):  
...         return 'Heads!'  
...     else:  
...         return 'Tails!'  
...     return 'The coin landed on its edge!'  
...  
>>> print(coinFlip())  
Tails!
```

The `return 'The coin landed on its edge!'` line is dead code because the code in the `if` and `else` blocks returns before the execution could ever reach that